



降

维



TRANSWARP
星环科技



目录

CONTENTS

- 1 特征降维概述
- 2 主成分分析
- 3 线性判别分析

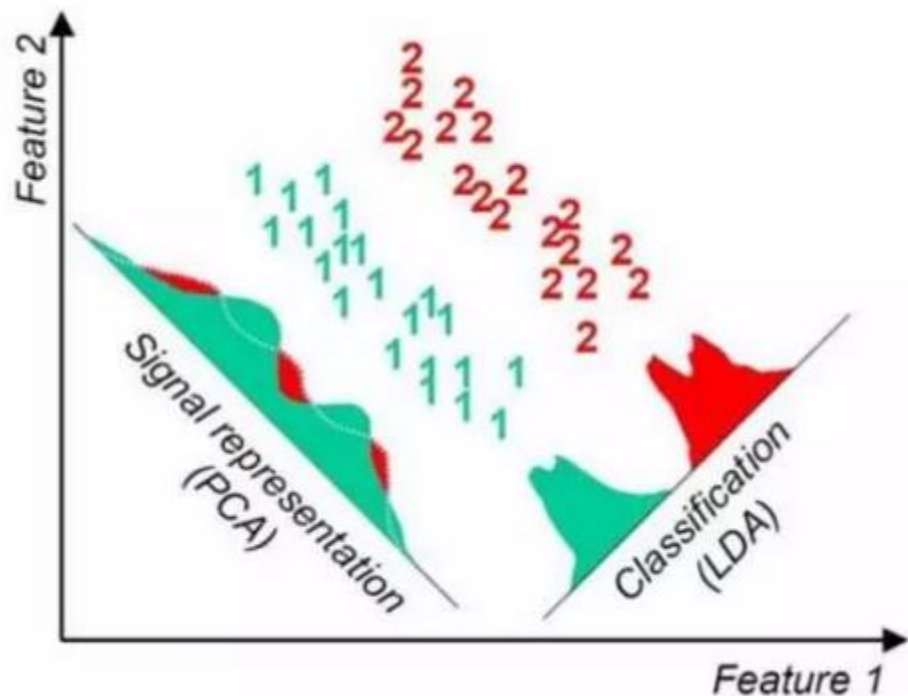


1 chapter

特征降维概述

特征降维概述

- 特征降维一般在特征选择做完以后进行。
- 常用降维方法：
 - 主成分分析法 (PCA)：无监督方法，映射后的样本具有更大的**发散性**
 - 线性判别分析法(LDA)：有监督方法，映射后的样本有最好的**分类性能**

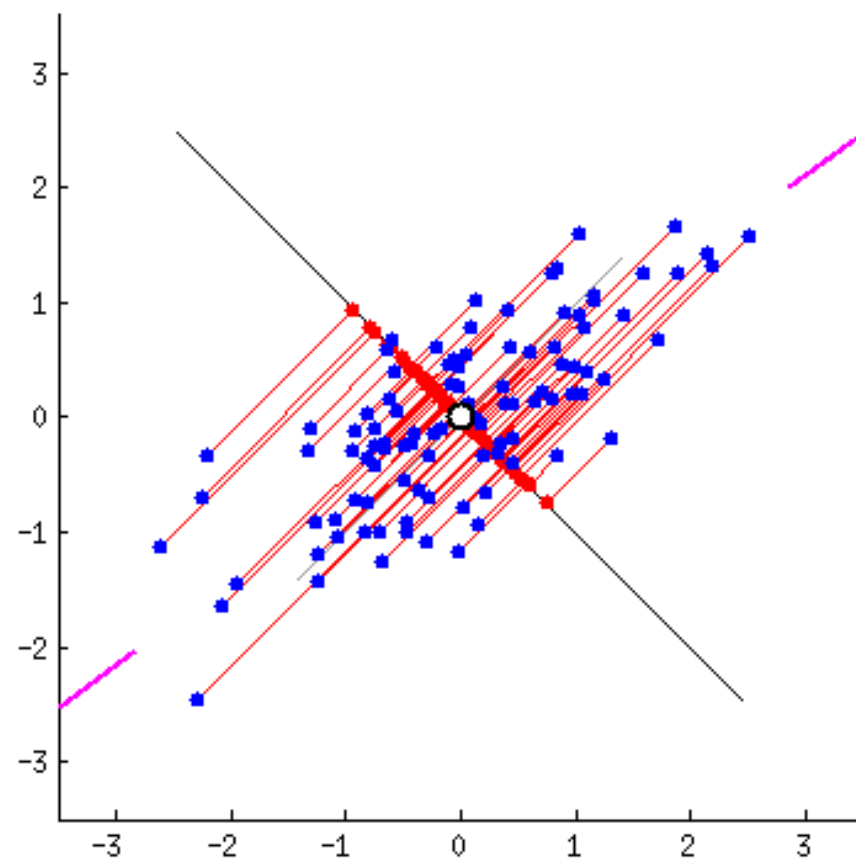


1
2
chapter

主成分分析

PCA-概述

- **PCA (Principal Component Analysis)** 是一种常见的数据分析方式，常用于高维数据的降维，可用于提取数据的**主要特征分量**。
- PCA 的数学推导：
 - **最大可分性**：映射后方差最大
 - **最近重构性**：点到映射平面距离最小



PCA-先导知识

- 向量内积
- 基
- 基变换
- 去均值
- 特征值和特征向量

向量内积

- 两个向量的 A 和 B 内积：

$$(a_1, a_2, \dots, a_n) \cdot (b_1, b_2, \dots, b_n)^T = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

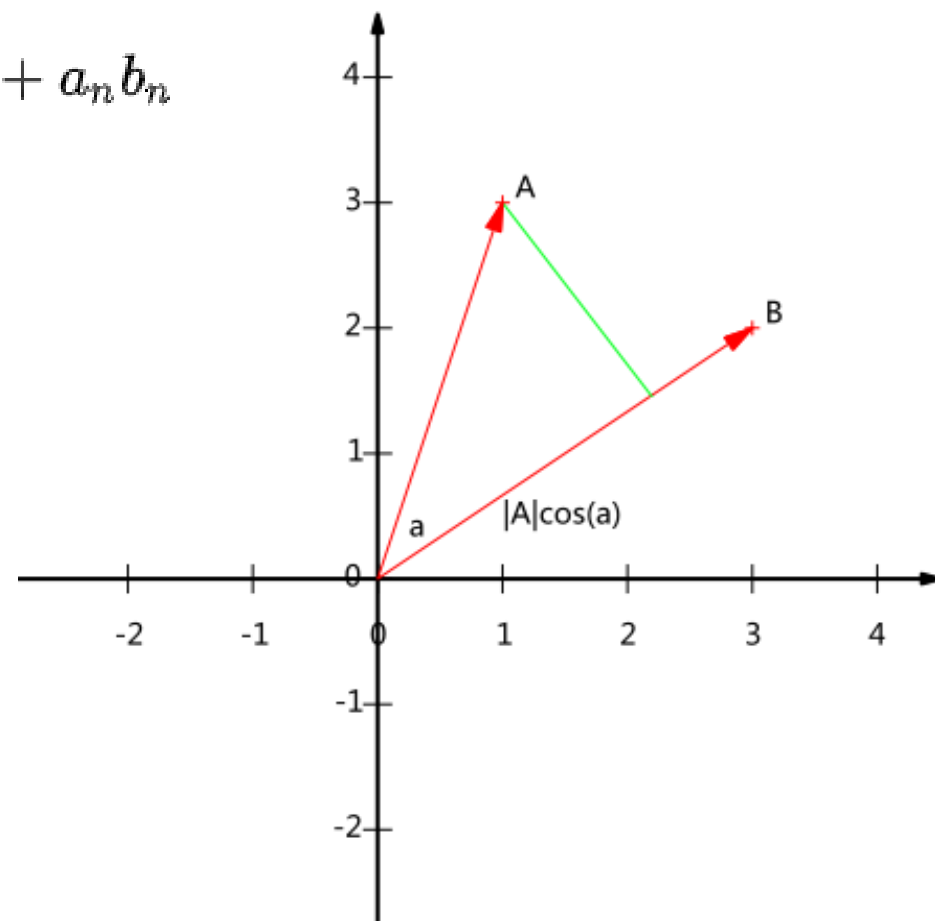
- 几何角度：为了简单起见，我们假设 A 和 B 均为二维向量

$$A = (x_1, y_1), \quad B = (x_2, y_2) \quad A \cdot B = |A||B|\cos(\alpha)$$

- 如果假设 B 的模为 1，即让 $|B| = 1$ ，那么就变成了：

$$A \cdot B = |A|\cos(a)$$

A 与 B 的内积值等于 A 向 B 所在直线投影的标量大小。

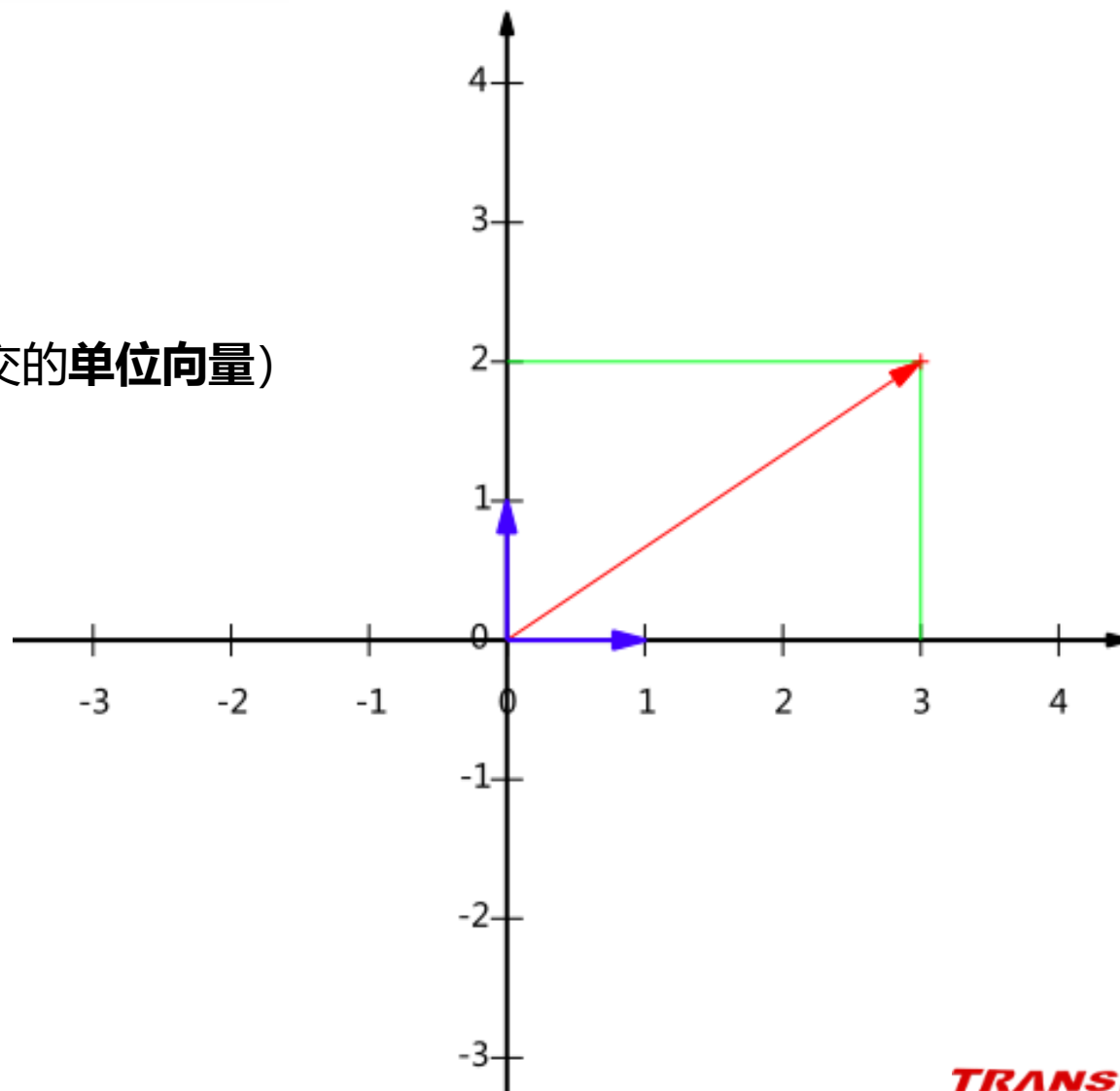


基

- 向量 (x,y) 实际上表示线性组合：

$$x(1, 0)^T + y(0, 1)^T$$

$(1,0)$ 和 $(0,1)$ 叫做二维空间中的一组**基**（彼此正交的**单位向量**）



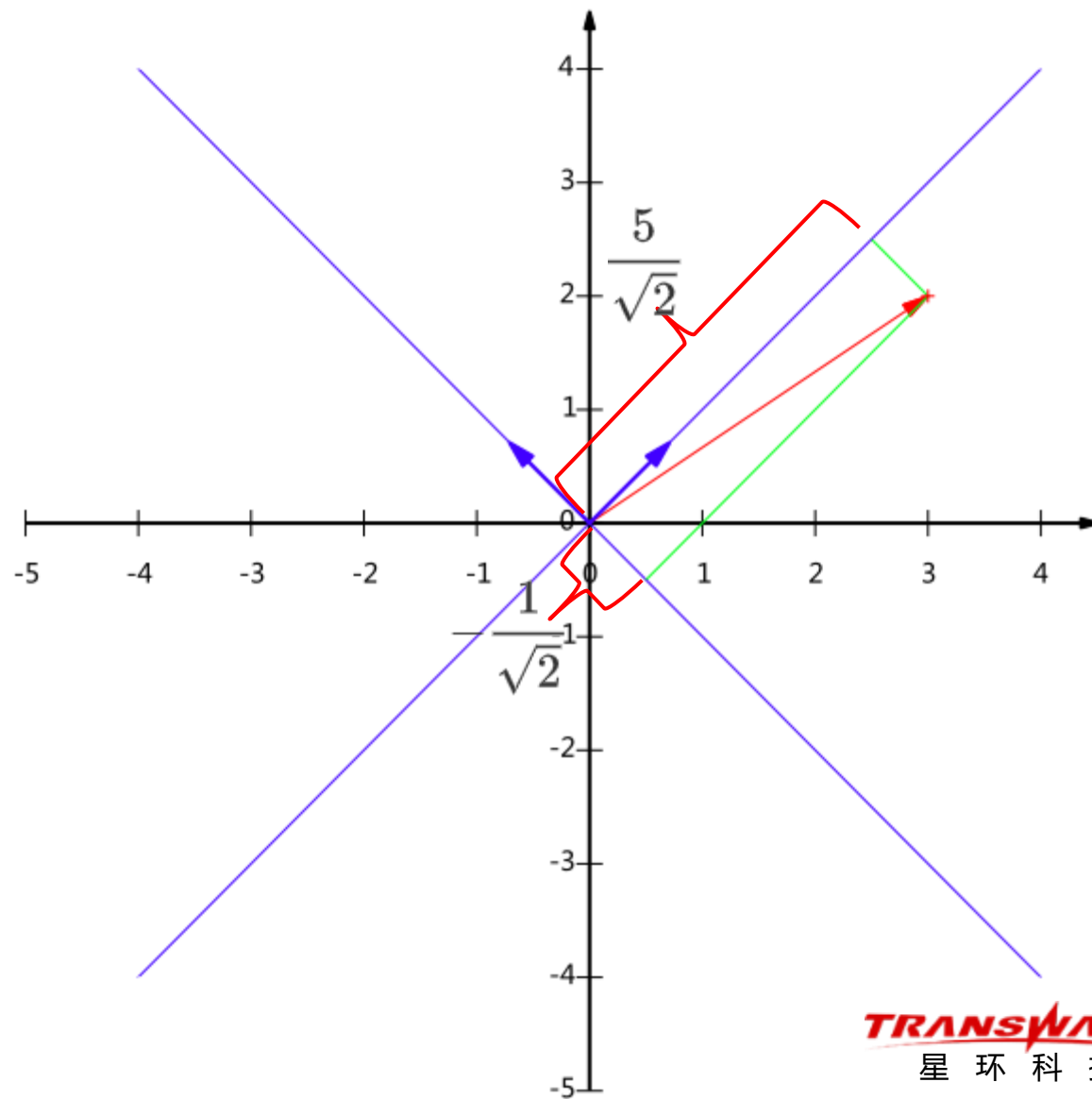
基变换

- $(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$ 和 $(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})$ 也可以成为一组基

如何计算(3,2)在新基（新坐标系）上的坐标？

根据内积的几何意义，我们只要分别计算(3,2)和两个基的内积，得到向量在两个基上的投影值，就可以得到新坐标。

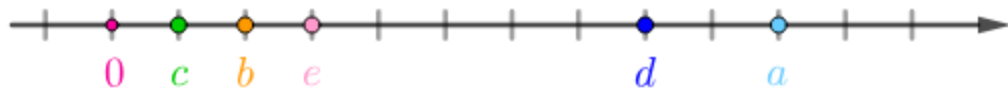
$$(\frac{5}{\sqrt{2}}, -\frac{1}{\sqrt{2}})$$



去均值 (demean)

- 去均值的目的是**简化方差的计算**

	房价(百万元)
<i>a</i>	10
<i>b</i>	2
<i>c</i>	1
<i>d</i>	7
<i>e</i>	3

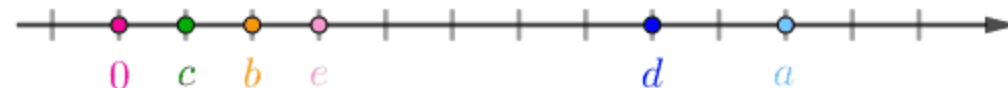


$$\bar{X} = \frac{X_1 + X_2 + X_3 + X_4 + X_5}{5} = \frac{10 + 2 + 1 + 7 + 3}{5} = 4.6$$

$$Var(X) = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$$

计算方差比较**麻烦**。

	房价(百万元)
<i>a</i>	$10 - \bar{X} = 5.4$
<i>b</i>	$2 - \bar{X} = -2.6$
<i>c</i>	$1 - \bar{X} = -3.6$
<i>d</i>	$7 - \bar{X} = 2.4$
<i>e</i>	$3 - \bar{X} = -1.6$

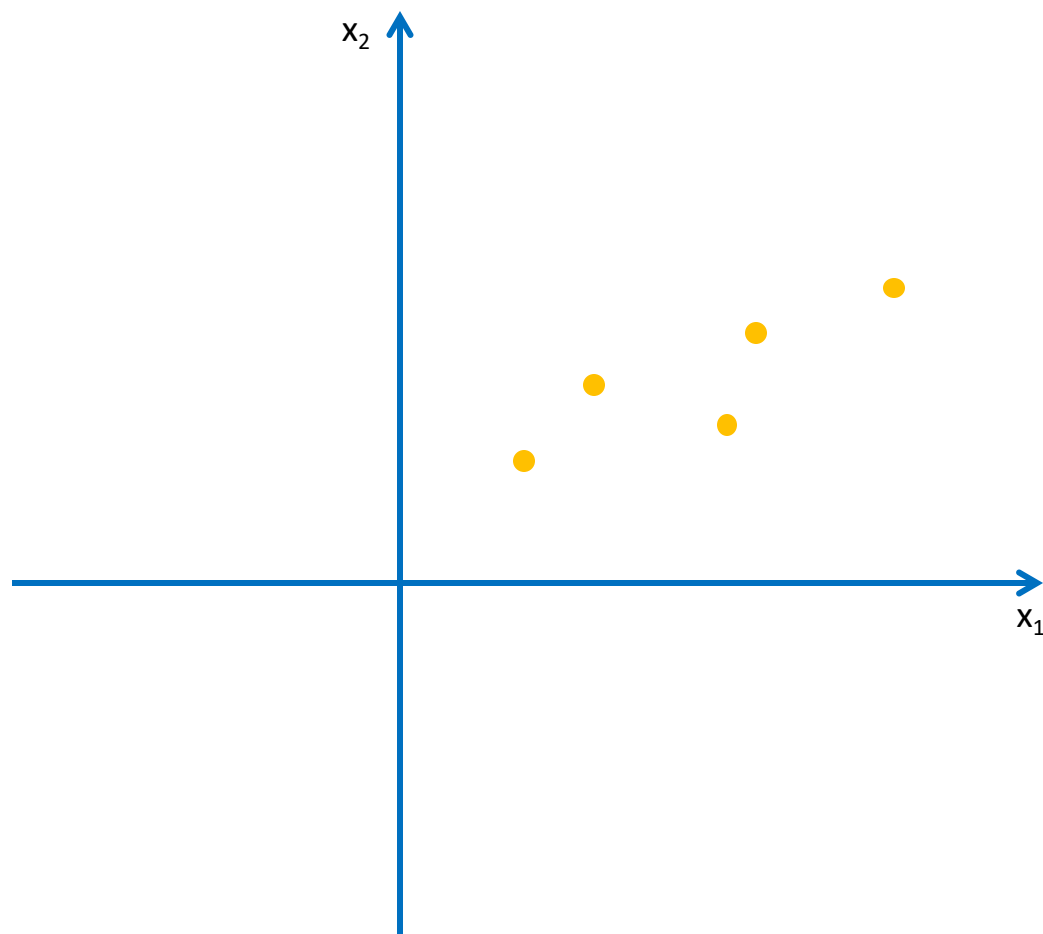


$$Var(X) = \frac{1}{n} (5.4^2 + (-2.6)^2 + (-3.6)^2 + 2.4^2 + (-1.6)^2)$$

使用**去均值后**的数据计算方差比较**简单**。

去均值 (demean)

- 去均值**不改变数据分布**



特征值和特征向量

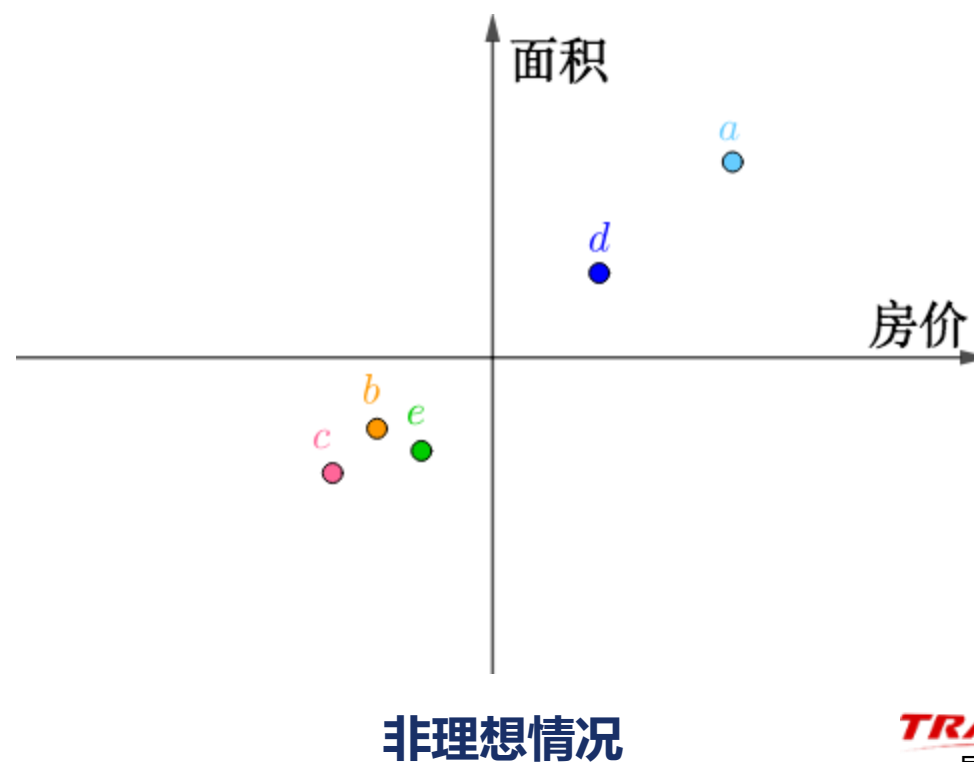
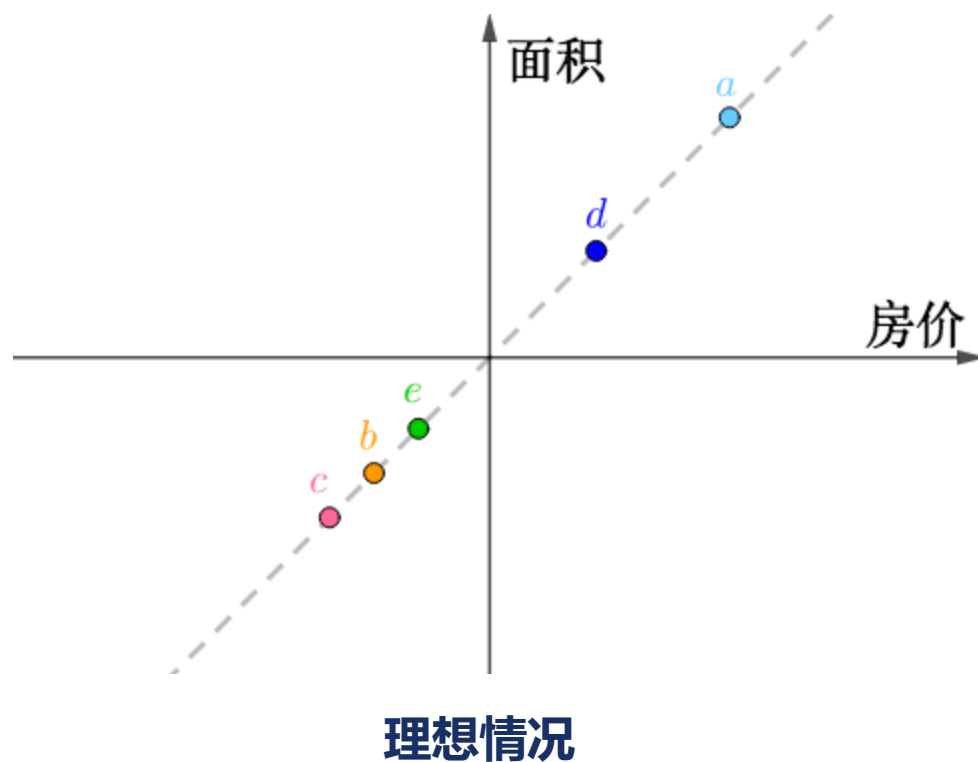
- 更多信息: <https://www.bilibili.com/video/BV1ys411472E?p=14>

$$A\mathbf{x} = \lambda\mathbf{x}$$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = \begin{bmatrix} \lambda x_1 \\ \lambda x_2 \\ \dots \\ \lambda x_n \end{bmatrix}$$

PCA-初步理解

- 理想情况下的降维
- 非理想情况下的降维



理想情况

- 假设有如下的数据:

	房价(百万元)	面积(百平米)
<i>a</i>	10	10
<i>b</i>	2	2
<i>c</i>	1	1
<i>d</i>	7	7
<i>e</i>	3	3

两列数据完全一样，是不是可以直接去掉一列
(特征选择)？还是进行PCA (特征降维)！

- 降维步骤:

- Step1: 去均值
- Step2: 旋转坐标轴 (求解特征向量)
- Step3: 计算数据点在新坐标系的坐标
(新的组合特征)
- Step4: 删除新坐标系中方差小的特征
(特征选择)

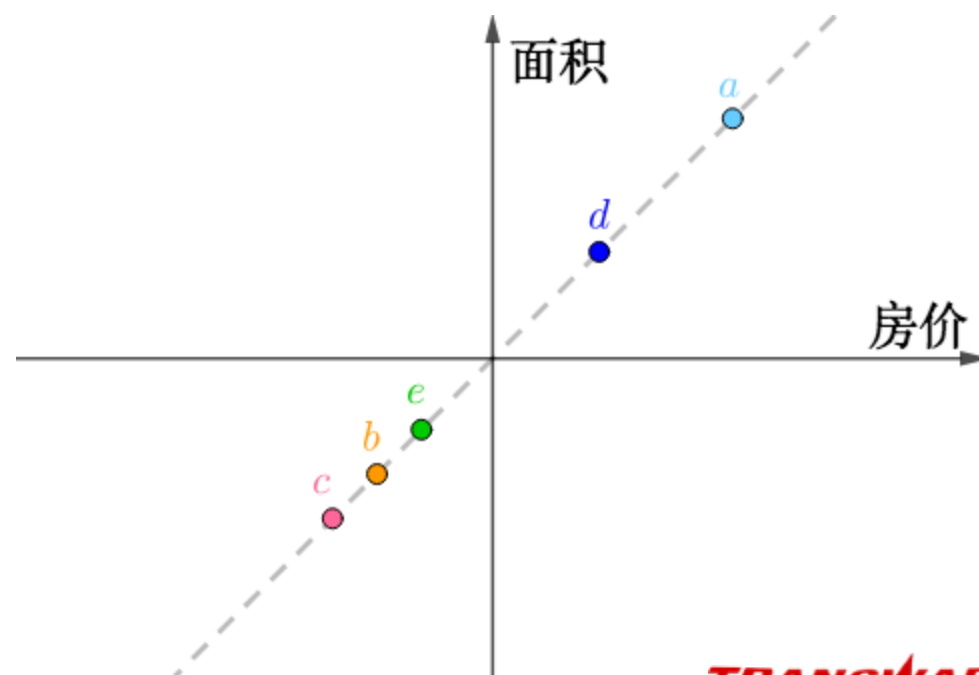
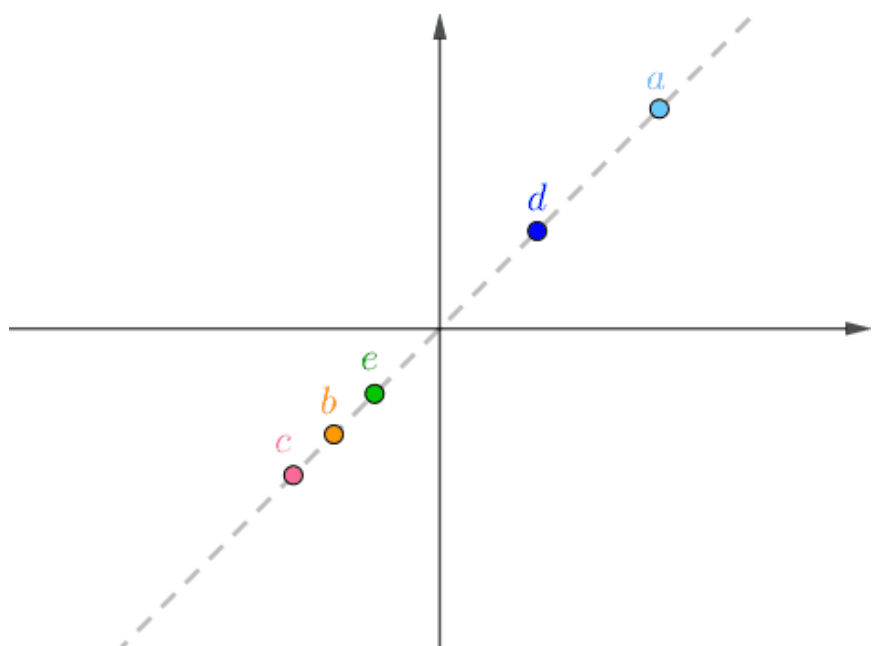
理想情况

	房价(百万元)	面积(百平米)
<i>a</i>	10	10
<i>b</i>	2	2
<i>c</i>	1	1
<i>d</i>	7	7
<i>e</i>	3	3

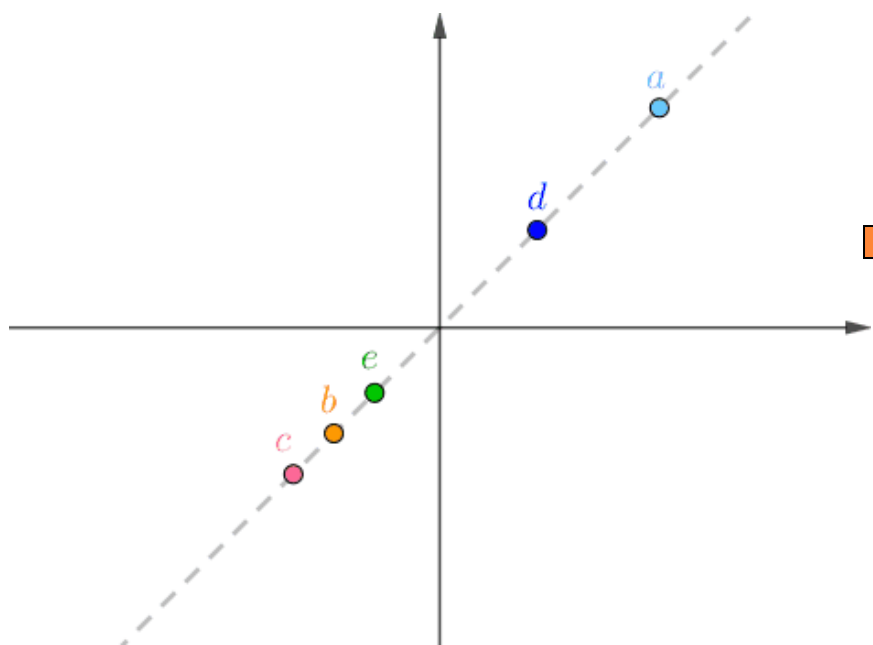
Step1: 去均值

	房价(百万元)	面积(百平米)
<i>a</i>	5.4	5.4
<i>b</i>	-2.6	-2.6
<i>c</i>	-3.6	-3.6
<i>d</i>	2.4	2.4
<i>e</i>	-1.6	-1.6

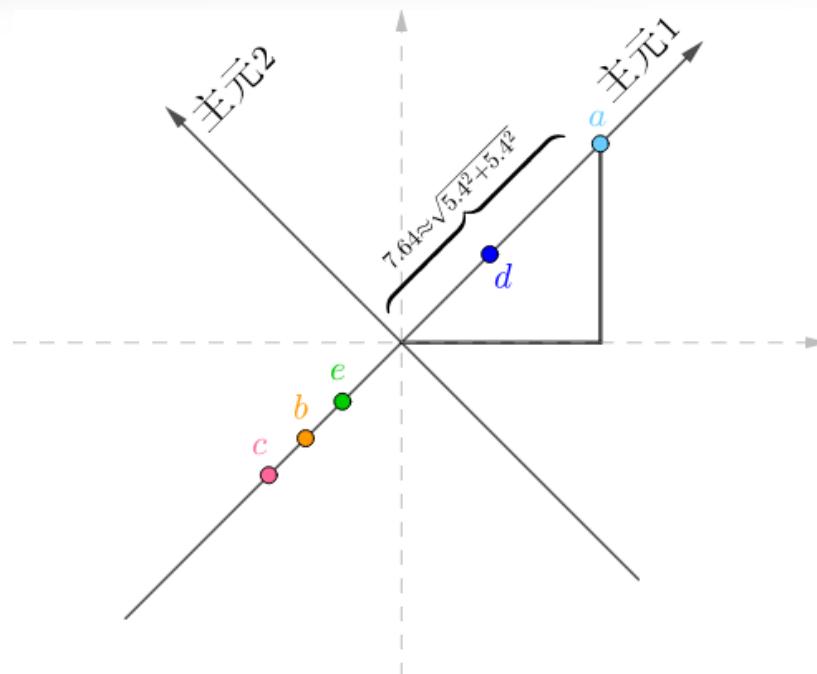
Step2: 旋转坐标轴



理想情况

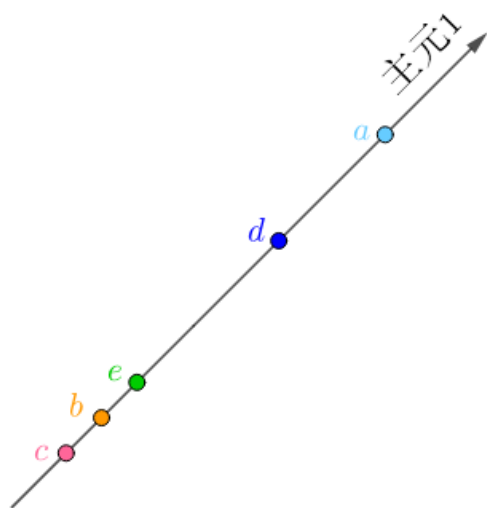


Step3: 计算新坐标

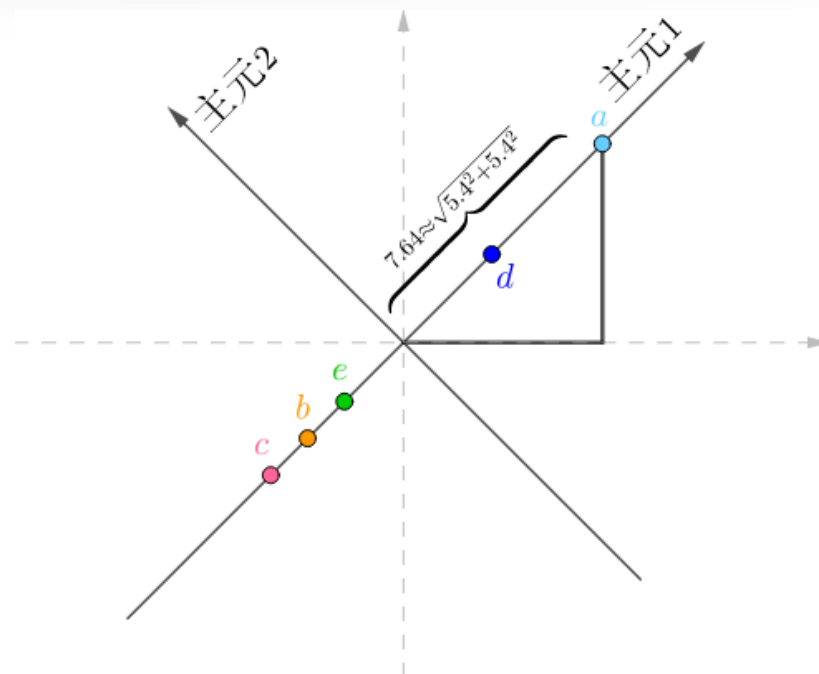


	主元1	主元2
<i>a</i>	7.64	0
<i>b</i>	-3.68	0
<i>c</i>	-5.09	0
<i>d</i>	3.39	0
<i>e</i>	-2.26	0

理想情况



	主元1
<i>a</i>	7.64
<i>b</i>	-3.68
<i>c</i>	-5.09
<i>d</i>	3.39
<i>e</i>	-2.26



Step4: 删除主元2



	主元1	主元2
<i>a</i>	7.64	0
<i>b</i>	-3.68	0
<i>c</i>	-5.09	0
<i>d</i>	3.39	0
<i>e</i>	-2.26	0

非理想情况

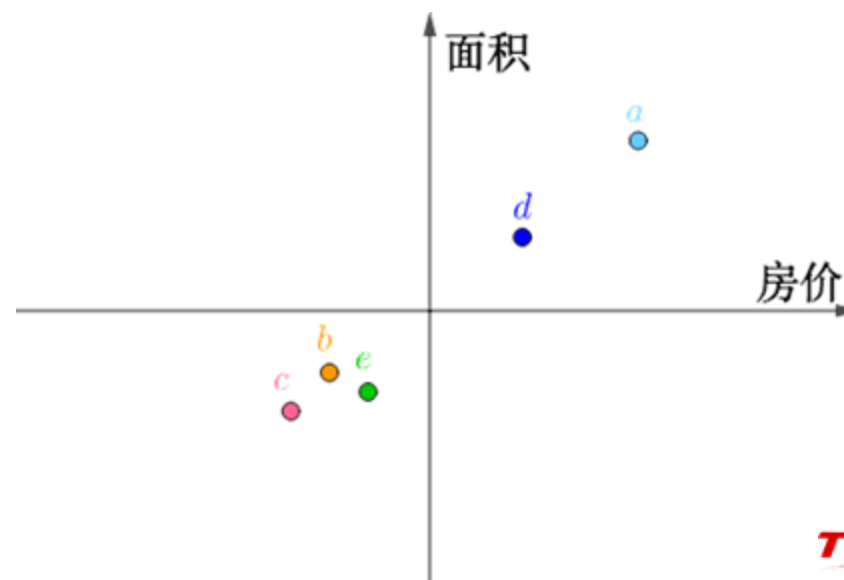
■ 现实中的数据:

	房价(百万元)	面积(百平米)
<i>a</i>	10	9
<i>b</i>	2	3
<i>c</i>	1	2
<i>d</i>	7	6.5
<i>e</i>	3	2.5

Step1: 去均值



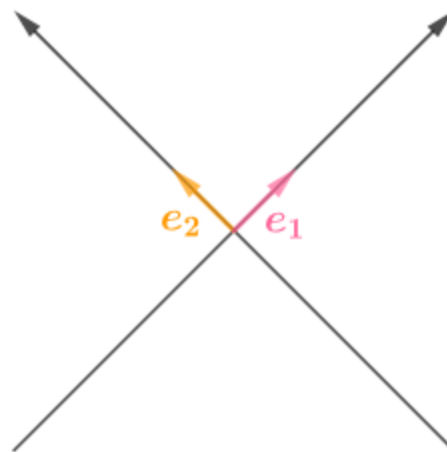
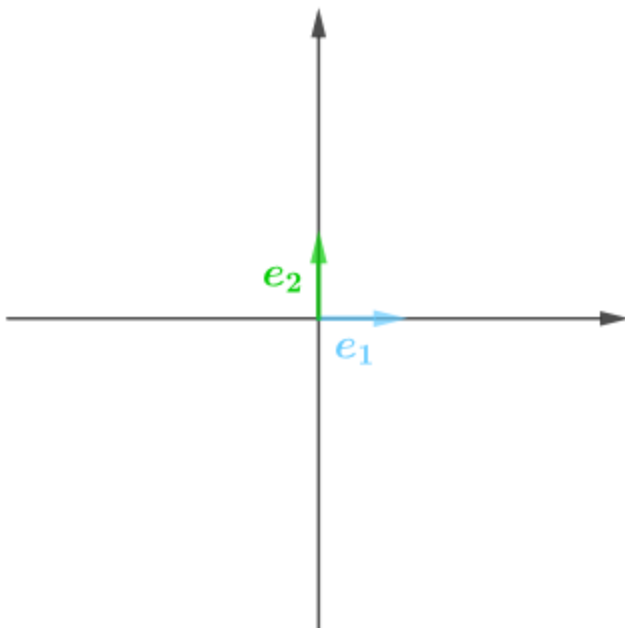
	房价(百万元)	面积(百平米)
<i>a</i>	5.4	4.4
<i>b</i>	-2.6	-1.6
<i>c</i>	-3.6	-2.6
<i>d</i>	2.4	1.9
<i>e</i>	-1.6	-2.1



该怎么旋转坐标轴呢?

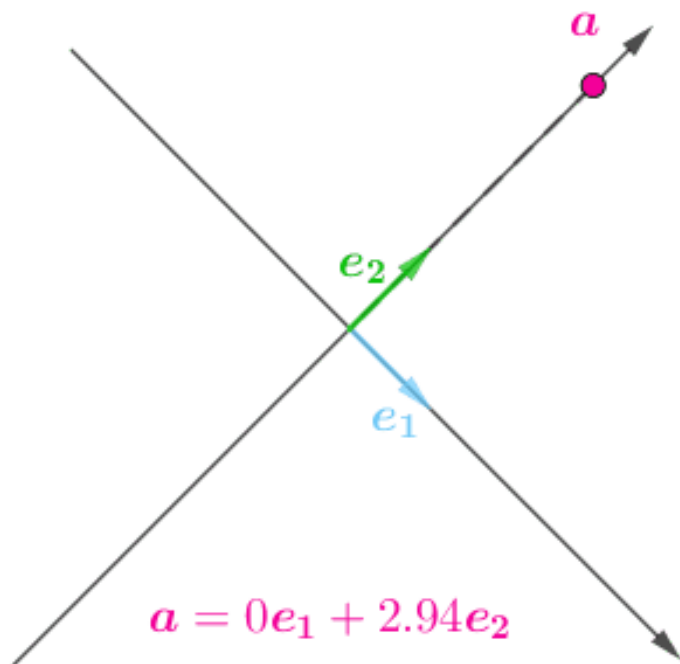
非理想情况

- 从线性代数的角度来看，二维坐标系总有各自的标准正交基（也就是**两两正交、模长为1**的基）：



非理想情况

- 一个点 $a = \begin{pmatrix} x \\ y \end{pmatrix}$, 表示在当前坐标系下标准正交基 e_1, e_2 的线性组合 $a = \begin{pmatrix} x \\ y \end{pmatrix} = xe_1 + ye_2$

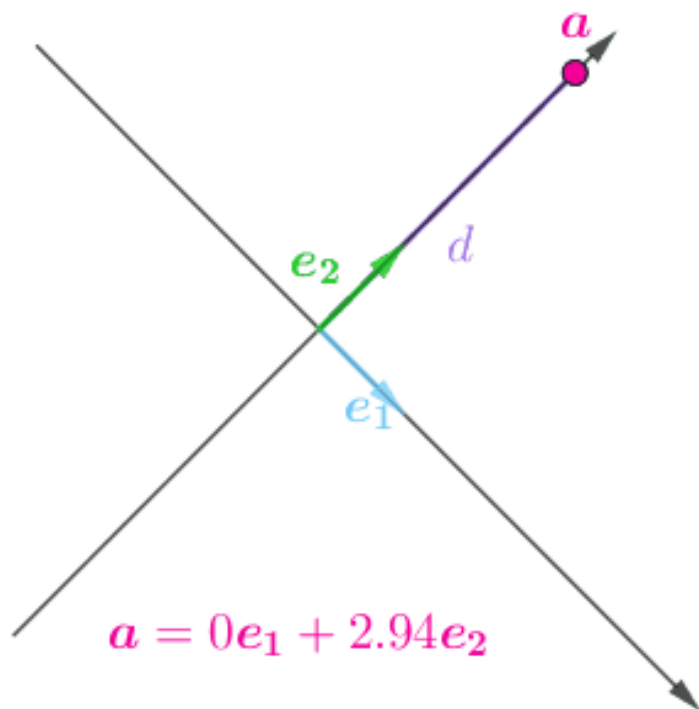


在不同坐标系中, x, y 的值会有所不同
(旋转的坐标表示不同的坐标系)

非理想情况

- **a到原点的距离d**不会因为坐标系改变而改变

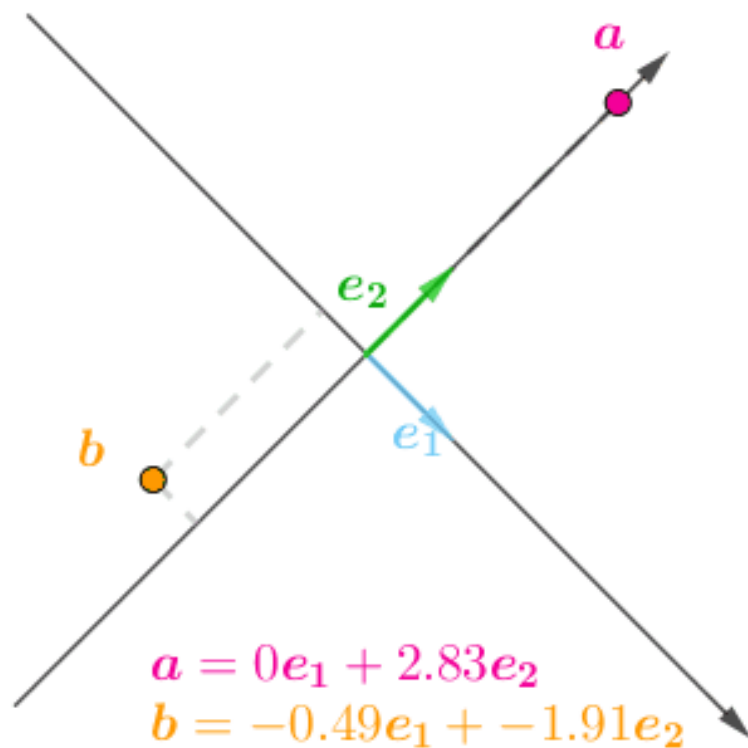
$$d^2 = x^2 + y^2$$



所以，在某坐标系下分配给**x较多**，那么分配给**y的就必然较少**，反之亦然。

非理想情况

- 为了尽量的**保留方差**，应该选择尽量多分配给 x_1, x_2 ，少分配给 y_1, y_2 的坐标系。



最大化 $X_1^2 + X_2^2 = \sum_{i=1}^2 X_i^2$

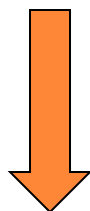
PCA-推导与求解

- 梯度上升法（最大可分性）
- 最大可分性
- 最近重构性

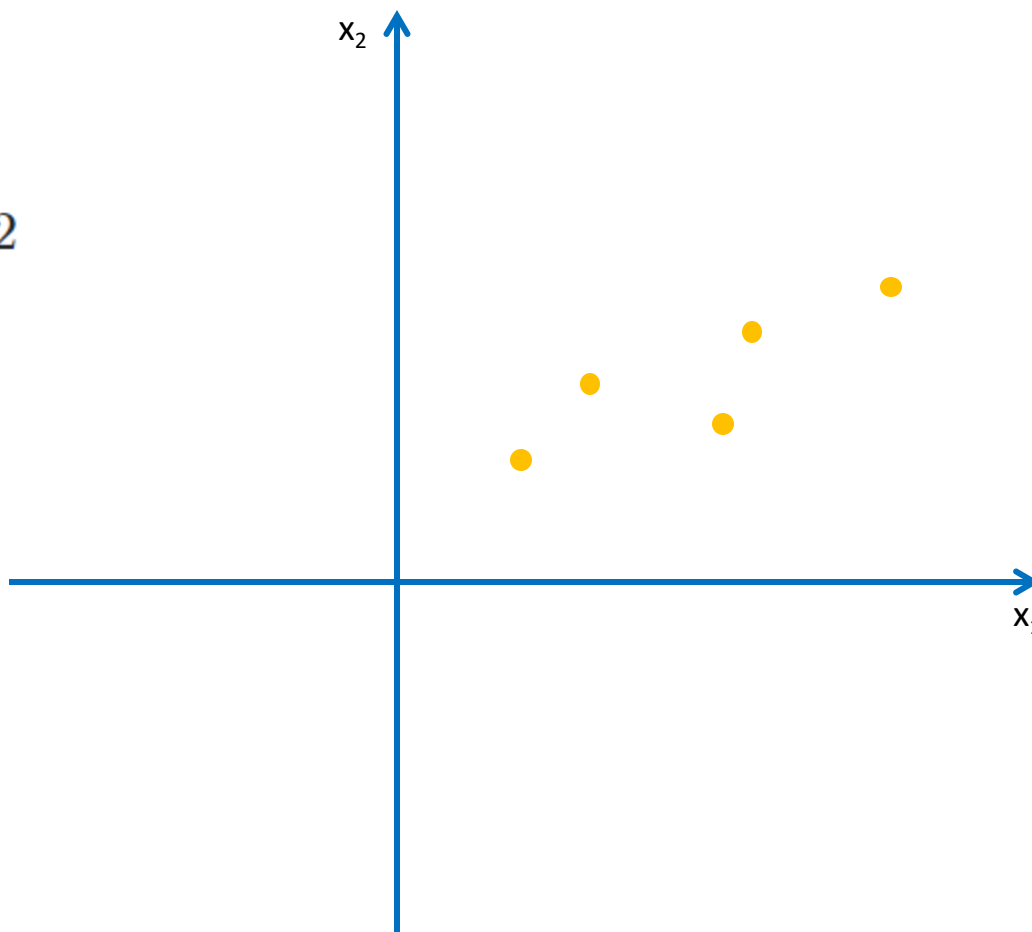
梯度上升法

- Step1: 去均值
- Step2: 构造优化目标函数（投影后方差最大化）
- Step3: 梯度上升法优化求解

$$Var(x) = \frac{1}{m} \sum_{i=1}^m (x_i - \bar{x})^2$$



$$Var(x) = \frac{1}{m} \sum_{i=1}^m (x_i)^2$$



梯度上升法

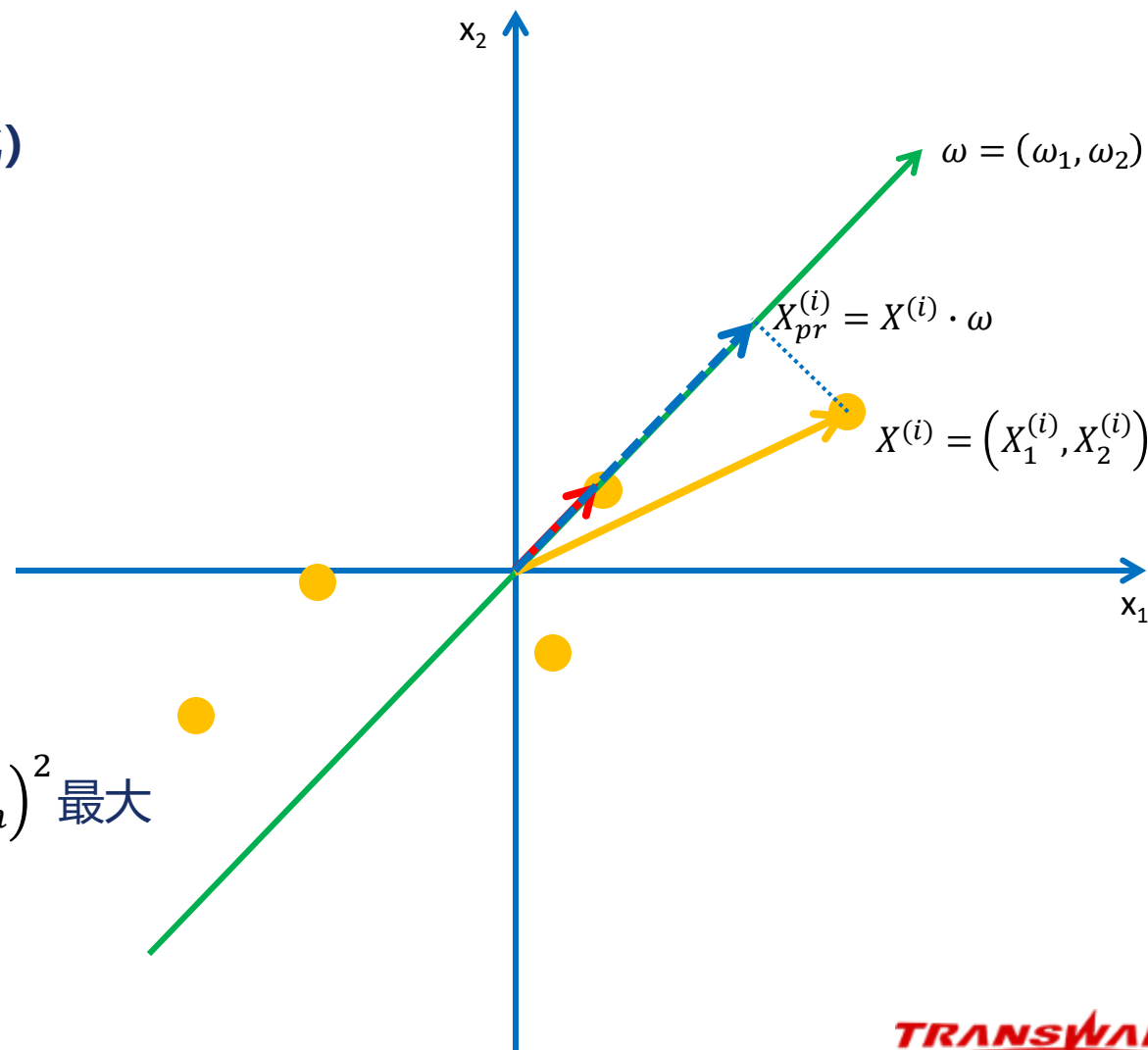
- Step1: 去均值
- Step2: 构造优化目标函数 (投影后方差最大化)

投影后方差为:

$$\begin{aligned} Var(X_{pr}) &= \frac{1}{m} \sum_{i=1}^m (X_{pr}^{(i)})^2 \\ &= \frac{1}{m} \sum_{i=1}^m (X^{(i)} \cdot w)^2 \end{aligned}$$

多维情况下, PAC 的目标函数:

$$\text{求 } w \text{ 使得 } f(w) = \frac{1}{m} \sum_{i=1}^m (X_1^{(i)} \cdot w_1 + X_2^{(i)} \cdot w_2 + \cdots + X_n^{(i)} \cdot w_n)^2 \text{ 最大}$$



梯度上升法

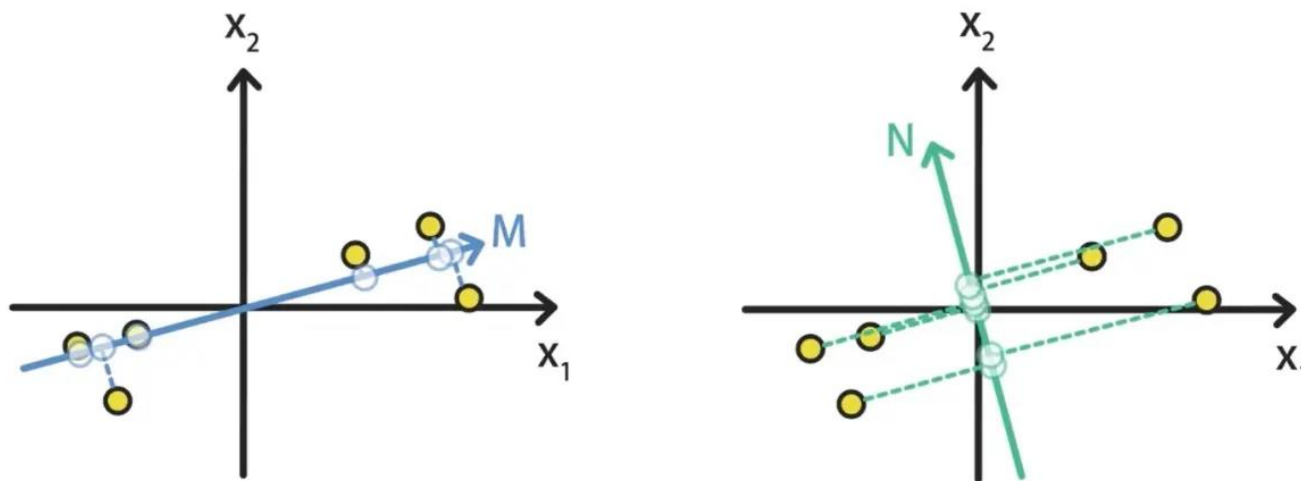
■ Step3: 梯度上升法优化求解

$$f(\omega) = \frac{1}{m} \sum_{i=1}^m \left(X_1^{(i)} \cdot \omega_1 + X_2^{(i)} \cdot \omega_2 + \dots + X_n^{(i)} \cdot \omega_n \right)^2$$

$$\begin{aligned} \nabla f &= \begin{pmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \dots \\ \frac{\partial f}{\partial w_n} \end{pmatrix} = \frac{2}{m} \begin{pmatrix} \sum_{i=1}^m (X_1^{(i)} w_1 + X_2^{(i)} w_2 + \dots + X_n^{(i)} w_n) X_1^{(i)} \\ \sum_{i=1}^m (X_1^{(i)} w_1 + X_2^{(i)} w_2 + \dots + X_n^{(i)} w_n) X_2^{(i)} \\ \dots \\ \sum_{i=1}^m (X_1^{(i)} w_1 + X_2^{(i)} w_2 + \dots + X_n^{(i)} w_n) X_n^{(i)} \end{pmatrix} \\ &= \frac{2}{m} \begin{pmatrix} \sum_{i=1}^m (X^{(i)} w) X_1^{(i)} \\ \sum_{i=1}^m (X^{(i)} w) X_2^{(i)} \\ \dots \\ \sum_{i=1}^m (X^{(i)} w) X_n^{(i)} \end{pmatrix} \left| \begin{pmatrix} \frac{2}{m} \cdot (X^{(1)} w, X^{(2)} w, X^{(3)} w, \dots, X^{(m)} w) \\ \begin{pmatrix} X_1^{(1)} & X_2^{(1)} & X_3^{(1)} & \dots & X_n^{(1)} \\ X_1^{(2)} & X_2^{(2)} & X_3^{(2)} & \dots & X_n^{(2)} \\ X_1^{(3)} & X_2^{(3)} & X_3^{(3)} & \dots & X_n^{(3)} \\ \dots & \dots & \dots & \dots & \dots \\ X_1^{(m)} & X_2^{(m)} & X_3^{(m)} & \dots & X_n^{(m)} \end{pmatrix} \end{pmatrix} \right. \\ &= \frac{2}{m} \cdot (Xw)^T \cdot X \\ &= \frac{2}{m} \cdot X^T (Xw) \end{aligned}$$

最近重构性

最近重构性是指样本到所投影的低维超平面的距离，要尽可能的小。



例如，平面上的样本到 M 的距离是蓝色线段，到 N 的距离是绿色线段。
所有蓝色线段的距离和，小于绿色线段，因此认为 M 比 N 好

最近重构性

■ 投影和重构

$$\|w_i\|_2 = 1, w_i^T w_j = 0 (i \neq j).$$

$$X = \begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 6 & 2 \\ 2 & 1 & 4 \end{pmatrix} \quad W = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{pmatrix} = \begin{pmatrix} -\frac{2}{3} & \frac{2}{3} & \frac{1}{3} \\ -\frac{1}{3} & -\frac{2}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{1}{3} & \frac{2}{3} \end{pmatrix}$$

投影

$$W^T \cdot X = \begin{pmatrix} -\frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & -\frac{2}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} & \frac{2}{3} \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 & 3 \\ 1 & 6 & 2 \\ 2 & 1 & 4 \end{pmatrix} = \begin{pmatrix} \frac{1}{3} & -\frac{8}{3} & 0 \\ \frac{2}{3} & -\frac{7}{3} & 2 \end{pmatrix}$$

特征列向量 $\quad$ 投影前特征 $\quad$ 投影后特征

$z_{i1} = w_{11}^T \cdot x_i \quad z_{i2} = w_{21}^T \cdot x_i$

$z_i = w^T x_i$

重构

$$W \cdot Z = \begin{pmatrix} -\frac{2}{3} & \frac{2}{3} \\ -\frac{1}{3} & -\frac{2}{3} \\ \frac{2}{3} & \frac{1}{3} \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{3} & -\frac{8}{3} & 0 \\ \frac{2}{3} & -\frac{7}{3} & 2 \end{pmatrix} = \begin{pmatrix} \frac{2}{9} & \frac{2}{9} & \frac{4}{3} \\ -\frac{5}{9} & \frac{22}{9} & -\frac{4}{3} \\ \frac{4}{9} & -\frac{23}{9} & \frac{2}{3} \end{pmatrix}$$

特征行向量 $\quad$ 重构前特征 $\quad$ 重构后特征

$\hat{x}_1 = \frac{1}{3} \cdot w_1 + \frac{2}{3} \cdot w_2 = \sum_{j=1}^2 z_{1j} \cdot w_j$

$\hat{x}_i = \sum_{j=1}^2 z_{ij} w_j = W \cdot Z_i$

最近重构性

■ 目标推导

$$z_i = W^T x_i \quad \hat{x}_i = \sum_{j=1}^2 z_{ij} w_j = W \cdot z_i$$

根据最近重构性，右式应被最小化，得到主成分分析的优化目标：

$$\min_{W'} -\text{tr}(W'^T X X^T W')$$

$$s.t. \quad W'^T W' = I.$$

利用拉格朗日乘子法，可得： $XX^T w_i = \lambda_i w_i$

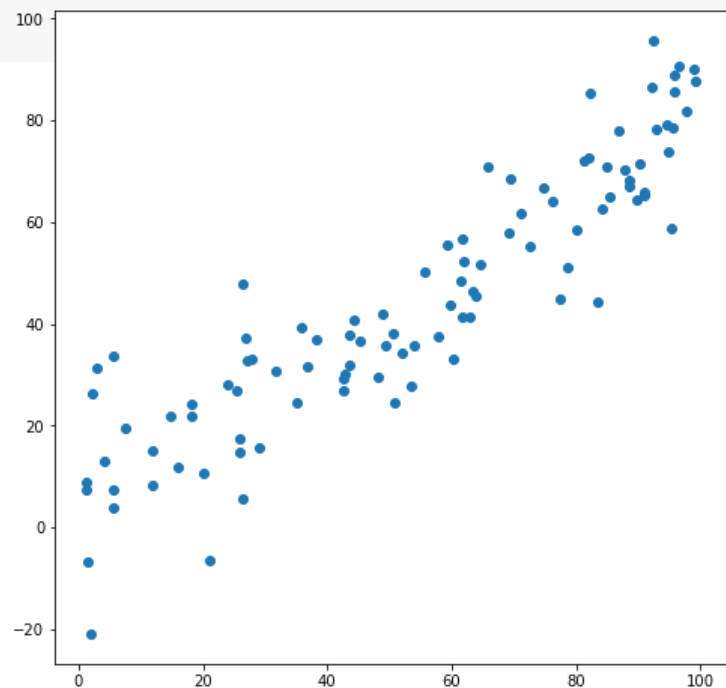
w_i 是协方差矩阵 XX^T 的特征向量

$$\begin{aligned} \sum_{i=1}^m \left\| \sum_{j=1}^{d'} z_{ij} w_j - x_i \right\|_2^2 &= \sum_{i=1}^m \|W' z_i - x_i\|_2^2 \\ &= \sum_{i=1}^m \|W' z_i\|_2^2 - 2 \sum_{i=1}^m (W' z_i)^T x_i + \sum_{i=1}^m \|x_i\|_2^2 \\ &= \sum_{i=1}^m z_i^T W'^T W' z_i - 2 \sum_{i=1}^m z_i^T x_i + \sum_{i=1}^m x_i^T x_i \\ &= -\sum_{i=1}^m z_i^T z_i + \sum_{i=1}^m x_i^T x_i \\ &= -\sum_{i=1}^m \text{tr}(z_i z_i^T) + \sum_{i=1}^m x_i^T x_i \\ &= -\text{tr}\left(\sum_{i=1}^m z_i z_i^T\right) + \sum_{i=1}^m x_i^T x_i \\ &= -\text{tr}\left(\sum_{i=1}^m W'^T x_i x_i^T W'\right) + \sum_{i=1}^m x_i^T x_i \\ &= -\text{tr}\left(W'^T \left(\sum_{i=1}^m x_i x_i^T\right) W'\right) + \sum_{i=1}^m x_i^T x_i \\ &= -\text{tr}(W'^T X X^T W') + \sum_{i=1}^m x_i^T x_i \end{aligned}$$

构造数据

```
import numpy as np
import matplotlib.pyplot as plt

x = np.empty((100, 2))    #x为100个样本，2个特征值的矩阵
x[:, 0] = np.random.uniform(0., 100., size = 100) #样本x的第1个特征值为均值为0，方差为100的100个值
x[:, 1] = 0.75*x[:, 0] + 3. + np.random.normal(0, 10., size=100) #样本x的第2个特征值和第1个特征值保持一定的线性关系，这里引入噪声。
plt.figure(figsize=(8, 8))
plt.scatter(x[:, 0], x[:, 1]) #将特征1和特征2绘制出来
plt.show()
```

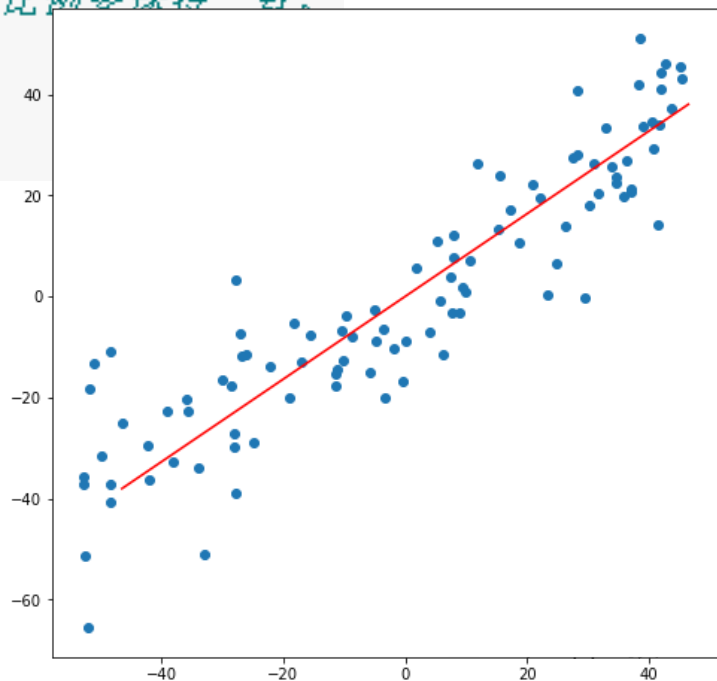


梯度上升法求解主成分

```
def demean(x):  
    return x-np.mean(x,axis=0) #首先对每个特征值进行去均值, axis=0代表按列求均值  
def f (w, x):  
    return np.sum((x.dot(w)**2))/len(x) #即方差公式, len(x)即样本的个数  
def df_math(w, x):  
    return x.T.dot(x.dot(w))*2./len(x) #即通过数学推导得出的求梯度公式  
def direstion(w):  
    return w /np.linalg.norm(w) #将w进行正则化, 因为w仅代表方向, 而每次求解后, w模不一定为1  
def gradietn_ascent(df, x, initial_w, eta, n_iters = 1e4, epsilon=1e-8): #梯度上升法, 传入初始w, 迭代次数和停止迭代的最小差值  
    w = direstion(initial_w) #先将传入的w变为单位向量  
    cur_iter = 0  
    while cur_iter < n_iters: #这里的思想和梯度下降法完全一致, 只是此时, eta为正, 即朝着函数值增大方向  
        gradient = df(w, x) #计算导数  
        last_w = w  
        w = w+eta*gradient #制度上升 梯度下降就是讲加号改为减号  
        w = direstion(w) #把w变为单位向量  
        if(abs(f(w, x)-f(last_w, x))<epsilon): #如果目标函数的值差距小于某个值, 则表示差不多已经到达极值点, 此时停止迭代  
            break  
    return w #就是我们要找的投影向量
```

梯度上升法求解主成分

```
1 x_mean = demean(x) #首先去均值处理 100, 2
2 #初始化不能为0向量, 因为0方向的向量并不代表某个具体方向,
3 #相当于没初始化w, 此外w的维度和x特征维度保持一致, 由上面推导的公式可以得出该结论。
4 initial_w = np.random.random(x.shape[1])
5 eta = 0.01 #初始化步长 学习率
6 #降维并得出第一个方向, 即第一个主成分, 使得在该方向上进行映射可以有最大的间距(方差)
7 w=gradietn_ascent(df_math, x_mean, initial_w, eta)
8 plt.figure(figsize=(8,8))
9 plt.scatter(x_mean[:,0], x_mean[:,1]) #首先将原始数据绘制
10 #将w表示的方向进行绘制, 为了更清楚地展示, 这里适当扩大w的表示范围, 但是w0和w1比例要保持一致。
11 plt.plot([-w[0]*60, w[0]*60], [-w[1]*60, w[1]*60], color = 'r')
12 plt.show()
13 print(w)
```



提取前n个主成分

```

1 def demean(x):
2     return x-np.mean(x,axis=0)
3 def f(w,x):
4     return np.sum((x.dot(w)**2))/len(x)
5 def df(w,x):
6     return x.T.dot(x.dot(w))*2./len(x)
7 def direstion(w):
8     return w /np.linalg.norm(w)
9 def first_component(x,initial_w,eta,n_iters = 1e4,epsilon=1e-8):
10    w = direstion(initial_w)
11    cur_iter =0
12    while cur_iter <n_iters:
13        gradient = df(w,x)
14        last_w = w
15        w = w+eta*gradient
16        w = direstion(w)
17        if(abs(f(w,x)-f(last_w,x))<epsilon):
18            break
19    return w
20 def first_n_componets(n,x,eta=0.01,n_iters =1e4,epsilon=1e-8):#这里将求前n个主成分进行封装
21    x_pca = x.copy() #首先将原始样本拷贝给x_pca
22    x_pca = demean(x_pca)
23    res =[] #用来存n个主成分
24    for i in range(n):
25        initial_w = np.random.random(x_pca.shape[1])
26        w = first_component(x_pca,initial_w,eta)
27        res.append(w)
28
29    x_pca = x_pca-x_pca.dot(w).reshape(-1,1)*w #之前用的是for循环,实际上可以进行向量化,向量化后可以优化计算速度
30    return res

```

```
1 first_n_componets(2,x)
```

```
[array([0.77844111, 0.62771764]), array([ 0.6277226 , -0.77843711])]
```

封装成类

```
1 import numpy as np
2
3 class PCA:
4
5     def __init__(self, n_components):
6         """初始化PCA"""
7         assert n_components >= 1, "n_components must be valid"
8         self.n_components = n_components #传入需要保留多少
9         self.components_ = None #用来保存找到的向量
10
11     def fit(self, X, eta=0.01, n_iters=1e4):
12         """获得数据集X的前n个主成分"""
13         assert self.n_components <= X.shape[1], \
14             "n_components must not be greater than the feature number of X"
15         def demean(X):
16             return X - np.mean(X, axis=0)
17         def f(w, X):
18             return np.sum((X.dot(w) ** 2)) / len(X)
19         def df(w, X):
20             return X.T.dot(X.dot(w)) * 2. / len(X)
21         def direction(w):
22             return w / np.linalg.norm(w)
```

封装成类

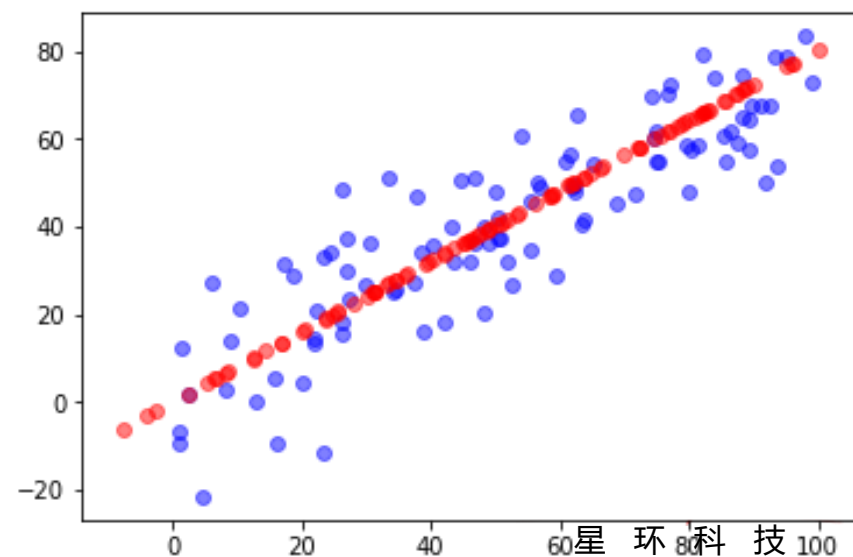
```
23 def first_component(X, initial_w, eta=0.01, n_iters=1e4, epsilon=1e-8):
24     w = direction(initial_w)
25     cur_iter = 0
26     while cur_iter < n_iters:
27         gradient = df(w, X)
28         last_w = w
29         w = w + eta * gradient
30         w = direction(w)
31         if (abs(f(w, X) - f(last_w, X)) < epsilon):
32             break
33         cur_iter += 1
34     return w
35 X_pca = demean(X)
36 self.components_ = np.empty(shape=(self.n_components, X.shape[1]))
37 for i in range(self.n_components):
38     initial_w = np.random.random(X_pca.shape[1])
39     w = first_component(X_pca, initial_w, eta, n_iters)
40     self.components_[i, :] = w
41     X_pca = X_pca - X_pca.dot(w).reshape(-1, 1) * w
42 return self
43 def transform(self, X):
44     """将给定的X, 映射到各个主成分分量中"""
45     assert X.shape[1] == self.components_.shape[1]
46     return X.dot(self.components_.T)
47 def inverse_transform(self, X):
48     """将给定的X, 反向映射回原来的特征空间"""
49     assert X.shape[1] == self.components_.shape[0]
50     return X.dot(self.components_)
51 def __repr__(self):
52     return "PCA(n_components=%d)" % self.n_components
```


调用封装的类

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x = np.empty((100, 2))
4 x[:, 0] = np.random.uniform(0., 100., size = 100)
5 x[:, 1] = 0.75*x[:, 0] + 3. + np.random.normal(0, 10., size=100) #这个数据集和上面使用的是一样的数据
6 pca = PCA(n_components=1)
7 pca.fit(x)
8 x_reduction = pca.transform(x) #降维
9 x_restore = pca.inverse_transform(x_reduction) #即将一维的数据重新映射回二维，仅仅是展示用
10 plt.scatter(x[:, 0], x[:, 1], color='b', alpha=0.5)
11 plt.scatter(x_restore[:, 0], x_restore[:, 1], color='r', alpha=0.5)
12 plt.show() #可以发现重新映射回原来矩阵形式时，只剩下主成分方向上的数据，restore的功能只是在高维的空间表达低维的数据
```

```
1 pca.components_
```

```
array([[0.77958382, 0.6262979 ]])
```

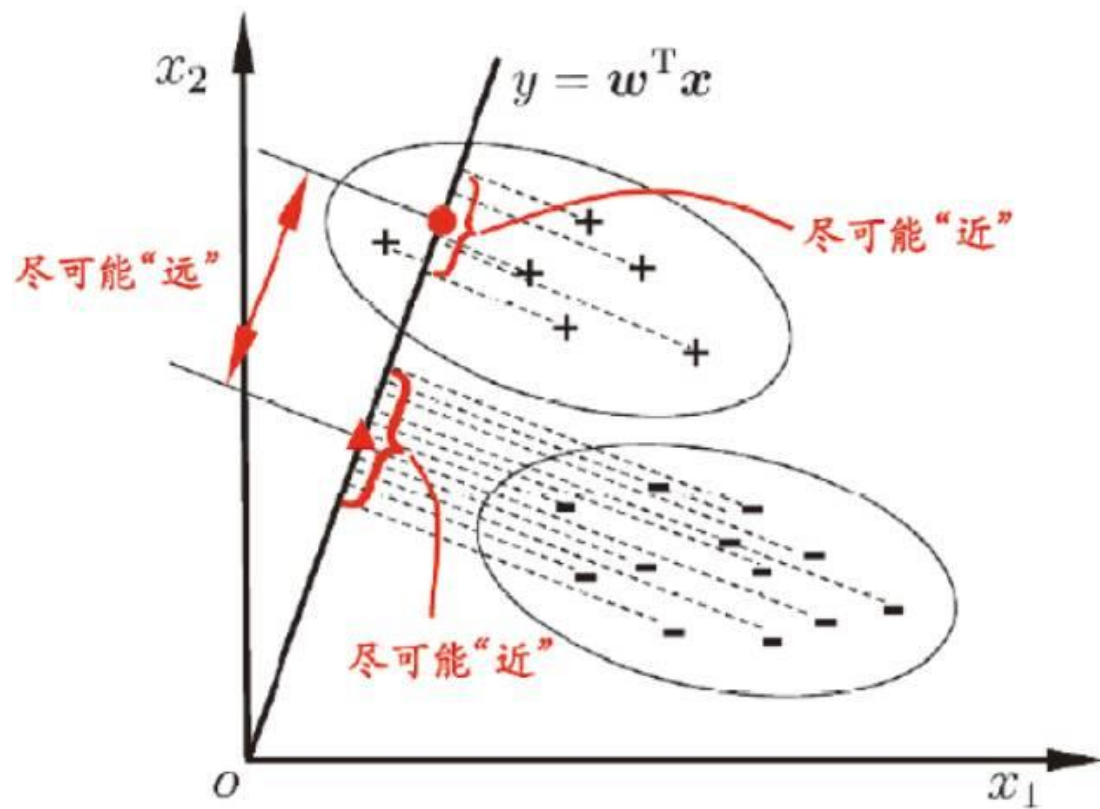


1
3
chapter

线性判别分析

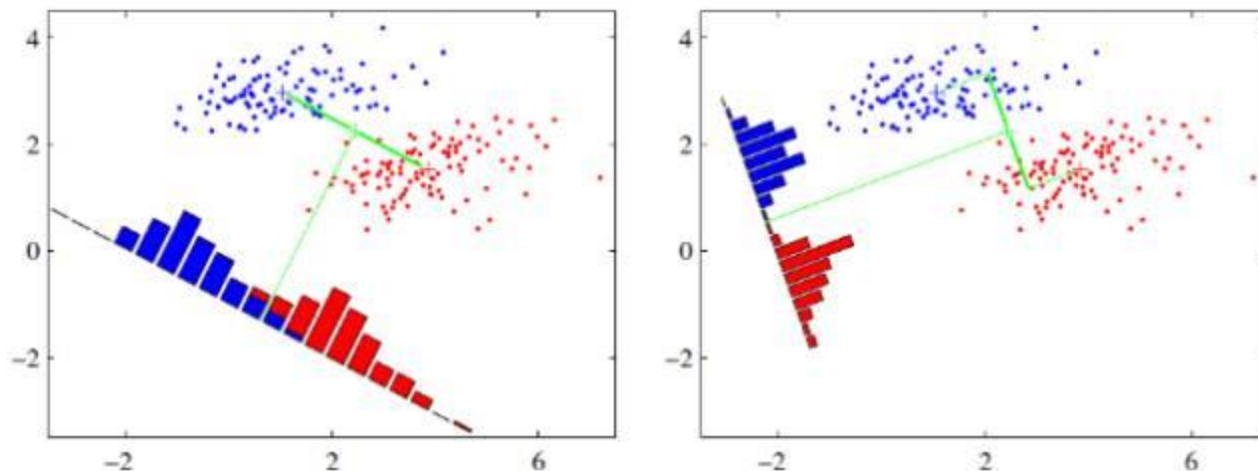
算法思想

- **Linear Discriminant Analysis, 简称LDA**, 一种监督学习的降维技术, 也就是说它的数据集的每个样本是有类别输出的, 这点和 PCA 不同。PCA 是不考虑样本类别输出的无监督降维技术。
- LDA 的思想可以用一句话概括, 就是 “**投影后类内方差最小, 类间方差最大**”。



示例

- 假设有两类数据，分别为红色和蓝色，如下图所示，这些数据特征是二维的，希望将这些数据投影到一维的一条直线上，让每一种类别数据的投影点尽可能的接近，而红色和蓝色数据中心之间的距离尽可能的大。



- 上图中提供了两种投影方式，哪一种能更好的满足我们的标准呢？从直观上可以看出，**右图要比左图的投影效果好，因为右图的蓝色数据和蓝色数据各个较为集中，且类别之间的距离明显。**左图则在边界处数据混杂。

算法原理

给定数据集 $\{(x_i, y_i)\}_{i=1}^m$

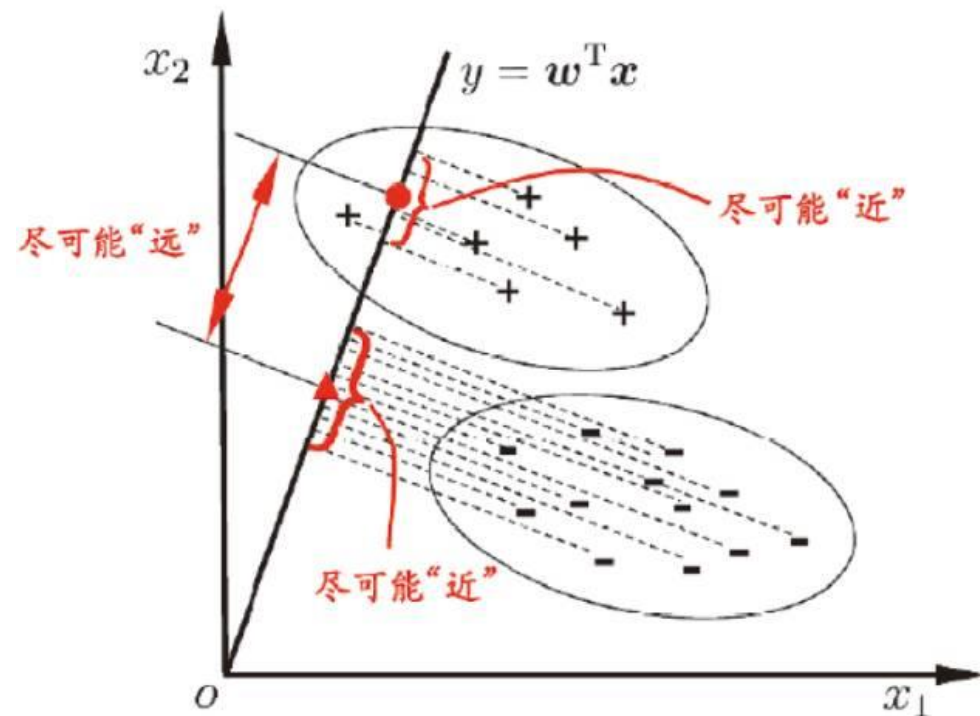
- 第 i 类样本的集合 X_i
- 第 i 类样本的均值向量 μ_i
- 第 i 类样本的协方差矩阵 Σ_i
- 两类样本的中心在直线上的投影： $w^T \mu_0$ 和 $w^T \mu_1$
- 两类样本的协方差 $w^T \Sigma_0 w$ 和 $w^T \Sigma_1 w$

同类样本的投影点尽可能**接近** $\rightarrow w^T \Sigma_0 w + w^T \Sigma_1 w$ 尽可能**小**

异类样本的投影点尽可能**远离** $\rightarrow \|w^T \mu_0 - w^T \mu_1\|_2^2$ 尽可能**大**

于是，**最大化**：

$$J = \frac{\|w^T \mu_0 - w^T \mu_1\|_2^2}{w^T \Sigma_0 w + w^T \Sigma_1 w} = \frac{w^T (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T w}{w^T (\Sigma_0 + \Sigma_1) w}$$



最大化广义瑞利熵 (generalized Rayleigh quotient)

给定数据集 $\{(x_i, y_i)\}_{i=1}^m$

- 类内散度矩阵 (within-class scatter matrix)

$$S_w = \Sigma_0 + \Sigma_1 = \sum_{x \in X_0} (x - \mu_0)(x - \mu_0)^T + \sum_{x \in X_1} (x - \mu_1)(x - \mu_1)^T$$

- 类间散度矩阵 (between-class scatter matrix)

$$S_b = (\mu_0 - \mu_1)(\mu_0 - \mu_1)^T$$

- LDA的目标: 最大化广义瑞利熵: $J = \frac{w^T S_b w}{w^T S_w w}$

拉格朗日乘子法求解

- 令 $w^T S_w w = 1$, 最大化广义瑞利熵等价形式为:
$$\min_w -w^T S_b w$$
$$s.t. w^T S_w w = 1$$

运用拉格朗日乘子法, 有 $S_b w = \lambda S_w w$

$S_b w$ 的反向恒为 $\mu_0 - \mu_1$, 不妨令 $S_b w = \lambda(\mu_0 - \mu_1)$

于是 $w = S_w^{-1}(\mu_0 - \mu_1)$

- 实践中通常是进行奇异值分解 $S_w = U \Sigma V^T$, 然后 $S_w^{-1} = V \Sigma^{-1} U^T$

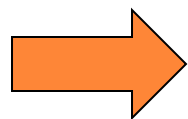
N个类的情况

- 全局散度矩阵 $S_t = S_b + S_w = \sum_{i=1}^m (x_i - \mu)(x_i - \mu)^T$
- 类内散度矩阵 $S_w = \sum_{i=1}^N S_{w_i}$ $S_{w_i} = \sum_{x \in X_i} (x - \mu_i)(x - \mu_i)^T$
- 类间散度矩阵 $S_b = S_t - S_w = \sum_{i=1}^N m_i(\mu_i - \mu)(\mu_i - \mu)^T$

多分类LDA有多种实现方法：采用 S_t , S_b , S_w 中的任何两个

例如,
$$\max_W \frac{\text{tr}(W^T S_b W)}{\text{tr}(W^T S_w W)}$$

$$W \in \mathbb{R}^{d \times (N-1)}$$



$$S_b W = \lambda S_w W$$

W 的闭式解是 $S_w^{-1} S_b$ 的 $N-1$ 个最大广义特征值所对应的特征向量组成的矩阵例如,

LDA最多只能降维到
N-1 (类别数-1) 维

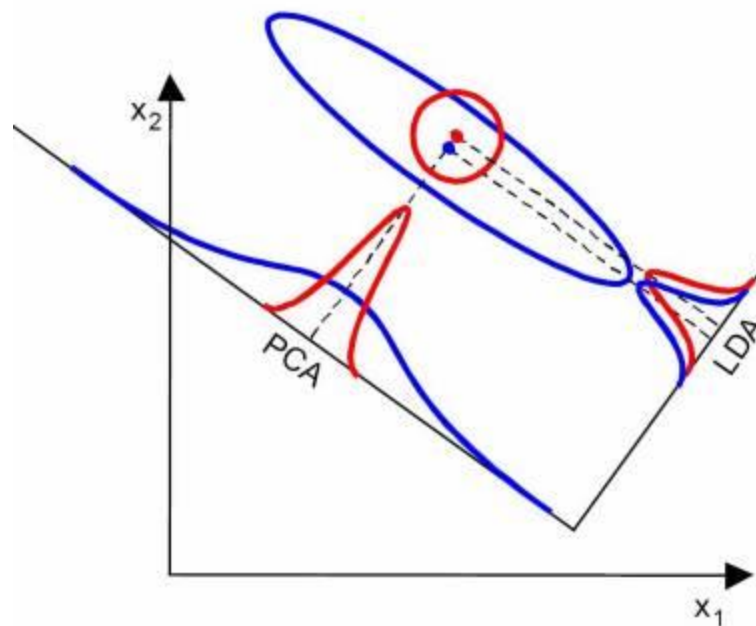
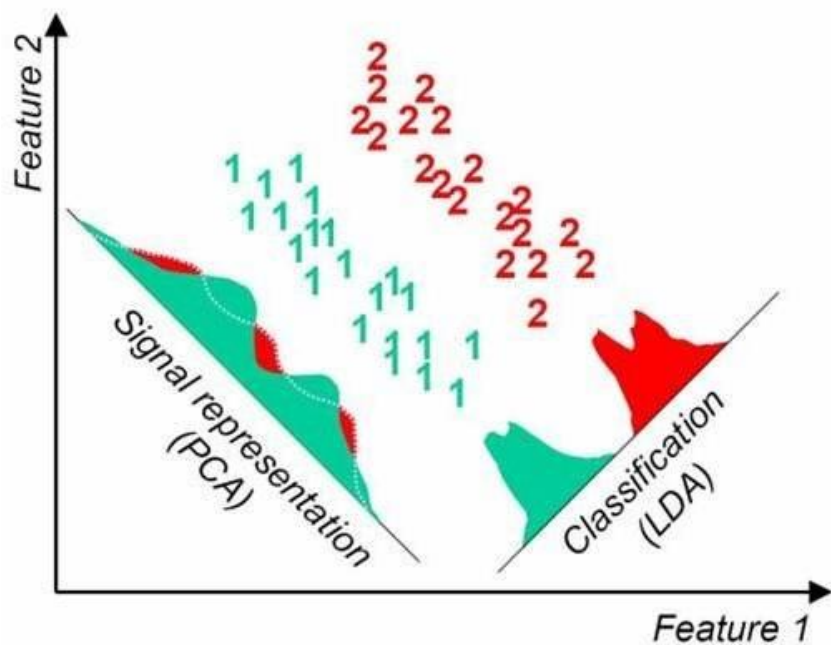
相同点

LDA 用于降维，和 PCA 有很多相同，也有很多不同的地方，因此值得好好的比较一下两者的降维异同点。

1. 两者均可以对数据进行降维。
2. 两者在降维时均使用了矩阵特征分解的思想。
3. 两者都假设数据符合高斯分布（PCA 对数据分布没有强假设，仅在高斯分布下表现最优）。

不同点

1. LDA 是有监督的降维方法，而 PCA 是无监督的降维方法
2. LDA 降维最多降到类别数 $k-1$ 的维数，而 PCA 没有这个限制。
3. LDA 除了可以用于降维，还可以用于分类。
4. LDA 选择分类性能最好的投影方向，而 PCA 选择样本点投影具有最大方差的方向。



加载手写数字识别数据

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn import datasets
4 digits = datasets.load_digits()
5 x = digits.data
6 y = digits.target
7 from sklearn.model_selection import train_test_split #这里依然需要区分训练数据集和测试数据集，不理解的可以参考knn博客
8 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state = 666)
```

降维前

```
1  #使用knn进行预测。  
2  from sklearn.neighbors import KNeighborsClassifier  
3  print(x.shape)  
4  knn = KNeighborsClassifier()  
5  knn.fit(x_train, y_train)  
6  knn.score(x_test, y_test)
```

(1797, 64)

0.987037037037037

LDA降维效果

```
1 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
2 lda = LinearDiscriminantAnalysis(n_components=9) #降到2维 lda是有监督的算法
3 # pca.fit(x_train)
4 x_train_reduction2 = lda.fit_transform(x_train, y_train)
5 print(x_train_reduction2.shape)
6 x_test_reduction2 = lda.transform(x_test) #测试数据集也必须降维
7 knn2 = KNeighborsClassifier()
8 knn2.fit(x_train_reduction2, y_train)
9 knn2.score(x_test_reduction2, y_test) #10分类 0.1
```

(1257, 9)

0.9648148148148148

PCA降维效果

```
1 from sklearn.decomposition import PCA
2 pca = PCA(n_components=9) #降到9维
3 # pca.fit(x_train)
4 x_train_reduction = pca.fit_transform(x_train)
5 print(x_train_reduction.shape)
6 x_test_reduction = pca.transform(x_test) #测试数据集也必须降维
7 knn = KNeighborsClassifier()
8 knn.fit(x_train_reduction, y_train)
9 knn.score(x_test_reduction, y_test) #10分类 0.1
```

(1257, 9)

0.9592592592592593



Q&A

TRANSWARP
星环科技